

GenAI Application Development: Phase 2

Study Guide: RAG & Data Preparation on Databricks

1. Select chunking strategy based on model & retrieval evaluation

Source: <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/quality-data-pipeline-rag> & https://docs.databricks.com/aws/en/sql/language-manual/functions/ai_prep_search

Core Concept: Chunking breaks down parsed text data into smaller, manageable pieces to optimize semantic retrieval. A well-designed chunking strategy balances context retention with computational efficiency.

How it works on Databricks: Databricks offers native SQL functions like `ai_prep_search()` which transforms structured outputs (from `ai_parse_document()`) into a format optimized for RAG. It splits content into semantic chunks and creates two representations: a `chunk_to_retrieve` (raw text) and a `chunk_to_embed` (context-enriched text combining the raw chunk with document-level context like titles and headers to improve semantic search).

When to use vs. alternatives:

- **Use Databricks built-in SQL functions:** When building scalable Lakeflow pipelines or standard RAG architectures over PDFs and Office documents, as this natively preserves structural context (headers, pages).
- **Alternatives:** Use Python libraries like LangChain's `RecursiveCharacterTextSplitter` when you need highly custom, token-level, or regex-based splitting strategies that the native semantic parser does not cover.

Common Mistakes: Creating chunks that are too large, which can exceed the model's context limit or include irrelevant information, reducing retrieval accuracy. Conversely, chunks that are too small lose the surrounding context required by the LLM to generate a coherent response.

Code Example & Function: Preparing parsed documents for vector search indexing via Databricks SQL.

```
-- Function: ai_prep_search()
-- Transforms parsed documents into an array of chunks optimized for
-- vector search and RAG. It automatically handles semantic splitting.
SELECT
    document_id,
    ai_prep_search(ai_parse_document(raw_content)) AS prepared_chunks
FROM raw_documents_table;
```

2. Select an embedding model context length

Source: <https://docs.databricks.com/aws/en/machine-learning/foundation-model-apis/supported-models> & <https://docs.databricks.com/gcp/en/ai-search/ai-search>

Core Concept: The embedding model converts chunked text into numerical vectors. You must select an embedding model with a context length that accommodates your largest chunk size while leaving room for the model's operational overhead.

How it works on Databricks: Databricks Foundation Model APIs host models like General Text Embedding (GTE) supporting 8192 token context windows, or models like Qwen3-Embedding-0.6B handling up to ~32K tokens. Once generated, these vectors are synced to Databricks AI Search (formerly Vector Search), which uses algorithms like Hierarchical Navigable Small World (HNSW) to retrieve similar chunks.

When to use vs. alternatives:

- **Use standard context models (e.g., 8192 tokens):** For general unstructured documents (memos, wikis). Databricks generally recommends keeping actual chunk sizes much smaller (e.g., ~1024 tokens) so that multiple retrieved chunks can fit simultaneously into the generator LLM's prompt window.
- **Use long-context embedding models (~32K tokens):** If your source documents are dense technical manuals or code repositories where semantic meaning spans thousands of tokens, and standard chunking would destroy the context.

Common Mistakes: Expecting cosine similarity natively without normalization. Databricks AI Search uses the L2 distance metric. If you require cosine similarity equivalents, you must normalize your data point embeddings before feeding them into the vector search algorithm.

Code Example & Function: Creating a Databricks AI Search index to compute embeddings.

```
from databricks.vector_search.client import VectorSearchClient

vsc = VectorSearchClient()

# Function: vsc.create_delta_sync_index()
# Creates a Vector Search index that automatically syncs with a Delta table.
# It computes embeddings for the specified source column using the selected model.
vsc.create_delta_sync_index(
    endpoint_name="rag_search_endpoint",
    source_table_name="main.rag.chunked_docs",
    index_name="main.rag.docs_index",
    pipeline_type="TRIGGERED",
    primary_key="chunk_id",
    embedding_source_column="chunk_to_embed", # Generated by ai_prep_search
    embedding_model_endpoint_name="databricks-bge-large-en"
)
```